FarmToFork – Phase 4 Final Project Report

COMP333 – Database Systems

Student: Sama Baraawi | ID: 1231022

Table of Contents

1. Project Overview
2. ER Diagram
3. Entities, Relationships, and Cardinalities
   3.1 Entities
   3.2 Relationships and Cardinalities
4. Mapping ER Diagram to Relational Schema
5. Normalization
   5.1 First Normal Form (1NF)
   5.2 Second Normal Form (2NF)
   5.3 Third Normal Form (3NF)
   5.4 Boyce–Codd Normal Form (BCNF)
6. System Implementation
   6.1 Technologies Used
   6.2 System Architecture
   6.3 CRUD Operations
   6.4 Transaction Management
   6.5 Data Integrity and Validation
   6.6 Reports Implementation
7. Database Implementation (SQL)
8. Required SQL Queries (20 Queries)
9. Demo Scenario
10. Conclusion
11. Future Work

1. Project Overview

FarmToFork is a database-driven management system developed for **GreenRoots Cooperative**, an organization that connects local farmers with restaurants, grocery stores, and food markets. The system supports the full agricultural supply chain by managing **purchasing and sales operations**, **inventory tracking**, and **product freshness** through expiry-date management.

This final phase represents the complete implementation of the FarmToFork system, integrating the conceptual and logical database design from **Phase 1 and Phase 2** with the functional prototype developed in **Phase 3**. The result is a fully operational application built using **JavaFX** for the user interface and **MySQL** as the backend database.

The system manages key entities including **Products, Categories, Warehouses, Branches, Clients, Farmers, Purchases, and Sales**. It provides full **CRUD operations** for master data, supports sales and purchase transactions with **automatic stock updates**, and includes a **Reports tab** that executes **20 analytical SQL queries** to assist in monitoring performance and supporting decision-making.

Project Scope and Objectives

The scope of the FarmToFork project focuses on developing a **reliable, efficient, and scalable** system that enables GreenRoots Cooperative to manage agricultural product distribution from **farm to fork** in an organized and data-driven manner.

Main Objectives:

- Record and manage **product purchases from farmers** and **sales to clients**.
- Track **product quantities, categories, and expiry dates** to ensure freshness.
- Support **multiple branches** and at least **one central warehouse**.
- Generate meaningful analytical reports, such as **monthly sales summaries**, **top-performing farmers**, and **low-stock alerts**.
- Ensure **data integrity**, proper **database normalization**, and ease of future maintenance and expansion.

2. ER Diagram

Figure 1 shows the final ER diagram of the database.

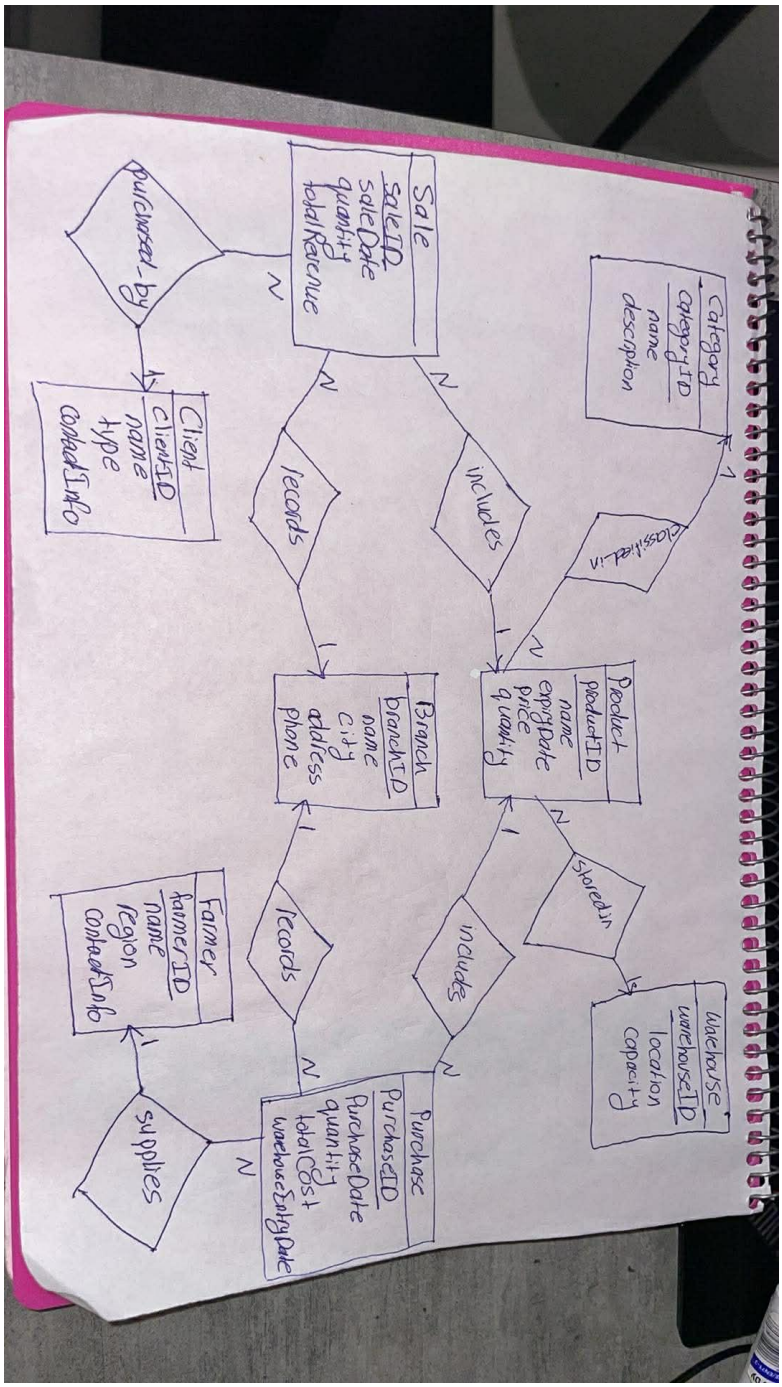


Figure 1 – FarmToFork ER Diagram

ER Diagram Design Considerations and Assumptions

The ER diagram was designed following the notations, rules, and modeling principles presented in the course. All entities in the system were modeled as strong entities, since each entity possesses a well-defined primary key that uniquely identifies its instances, and no entity is existence-dependent on another.

Only binary relationships were used in the design, and all relationships follow a one-to-many (1:N) cardinality. No many-to-many relationships were included, as all associations could be accurately represented using binary relationships without introducing redundancy or ambiguity. In addition, no ternary relationships were required, as the system logic did not impose constraints that could not be captured using binary relationships.

All entity participations in relationships were modeled as partial participation, since none of the entities are required to participate in a relationship at all times. Therefore, single lines were used to represent participation in all relationships.

Foreign key attributes were intentionally excluded from entity definitions in the ER diagram. Instead, relationships were explicitly represented using relationship sets, and foreign keys are introduced later during the ER-to-relational mapping. This approach avoids redundancy and aligns with conceptual-level modeling guidelines.

Derived attributes were not included in the ER diagram. For example, the unit price of a sale or purchase was omitted because it can be derived from other stored attributes such as quantity and total revenue or total cost. Including such attributes would introduce redundancy and violate normalization principles. These attributes may, however, appear in the relational schema or implementation for practical or performance-related reasons.

Attributes that represent real-world facts and cannot be derived from other attributes, such as quantity, total cost, total revenue, expiry date, and warehouse capacity, were retained in the ER diagram to preserve semantic clarity.

Relationship names were carefully chosen to clearly reflect the semantics of the domain, such as *classified_in*, *stored_in*, *supplies*, *records*, *includes*, and *purchased_by*. These names enhance the readability of the diagram and accurately describe the role of each relationship in the system.

Overall, the ER diagram represents a clean, normalized, and semantically accurate conceptual model of the FarmToFork system, fully consistent and suitable for conversion into a relational schema in the subsequent design phases.

3. Entities, Relationships, and Cardinalities

Entities: Category, Warehouse, Branch, Client, Farmer, Product, Purchase, Sale.

Key relationships (all are 1:N):

- Category (1) — (N) Product
- Warehouse (1) — (N) Product
- Branch (1) — (N) Sale
- Branch (1) — (N) Purchase
- Client (1) — (N) Sale
- Farmer (1) — (N) Purchase
- Product (1) — (N) Sale
- Product (1) — (N) Purchase

4. Mapping ER to Relational Schema

- **Relational schema (PK and FK are indicated):**
  - **CATEGORY**(*categoryID* **PK**, name, description)
  - **WAREHOUSE**(*warehouseID* **PK**, location, capacity)
  - **BRANCH**(*branchID* **PK**, name, city, address, phone)
  - **CLIENT**(*clientID* **PK**, name, type, contactInfo)
  - **FARMER**(*farmerID* **PK**, name, region, contactInfo)
  - **PRODUCT**(*productID* **PK**, name, expiryDate, price, quantity, categoryID **FK** → CATEGORY.categoryID,warehouseID **FK** → WAREHOUSE.warehouseID)
  - **PURCHASE**(*purchaseID* **PK**, purchaseDate, quantity, totalCost, warehouseEntryDate, farmerID **FK** → FARMER.farmerID,productID **FK** → PRODUCT.productID, branchID **FK** → BRANCH.branchID)
  - **SALE**(*saleID* **PK**, saleDate, quantity, totalRevenue,clientID **FK** → CLIENT.clientID, productID **FK** → PRODUCT.productID,branchID **FK** → BRANCH.branchID)

All relations in the relational schema were derived directly from the ER diagram, following the defined entities, relationships, and cardinality constraints, without introducing any additional relations or attributes.

Although the unit price may appear in the implementation phase for practical or performance-related reasons, it was intentionally excluded from the relational schema. This is because both **totalCost** and **totalRevenue** can be derived from the stored quantity and unit price values. Excluding derived attributes from the conceptual and relational design avoids redundancy and ensures consistency with normalization principles.

5. Normalization (1NF → 3NF/BCNF)

This section follows the normalization rules discussed in Chapter 7. First Normal Form requires atomic values , Second Normal Form requires full functional dependency of every non-prime attribute on the primary key , and BCNF requires that for every non-trivial FD $A \rightarrow B$, Amust be a superkey Summary by table:

Table	Functional Dependencies (FDs)	1NF	2NF / 3NF	BCNF & Notes
CATEGORY	categoryID → name, description	All attributes are atomic.	Single-attribute primary key; no partial or transitive dependencies.	Satisfies BCNF.
WAREHOUSE	warehouseID → location, capacity	All attributes are atomic.	Single-attribute primary key; no partial or transitive dependencies.	Satisfies BCNF.
BRANCH	branchID → name, city, address, phone	All attributes are atomic.	Single-attribute primary key; no partial or transitive dependencies.	Satisfies BCNF.
CLIENT	clientID → name, type, contactInfo	All attributes are atomic.	Single-attribute primary key; no partial or transitive dependencies.	Satisfies BCNF.
FARMER	farmerID → name, region, contactInfo	All attributes are atomic.	Single-attribute primary key; no partial or transitive	Satisfies BCNF.

			dependencies.	
PRODUCT	productID → name, expiryDate, price, quantity, categoryID, warehouseID	All attributes are atomic.	No partial or transitive dependencies; foreign keys reference other relations.	Satisfies BCNF.
SALE	saleID → saleDate, quantity, totalRevenue, clientID, productID, branchID	All attributes are atomic.	No partial or transitive dependencies.	Satisfies BCNF. totalRevenue stored for reporting; consistency enforced by application logic.
PURCHASE	purchaseID → purchaseDate, quantity, totalCost, warehouseEntryDate, farmerID, productID, branchID	All attributes are atomic.	No partial or transitive dependencies.	Satisfies BCNF. totalCost stored for reporting; consistency enforced by application logic.

Note: totalCost is stored as a transaction value to support reporting; consistency is ensured by the application/transaction logic during inserts and updates.

6. System Implementation

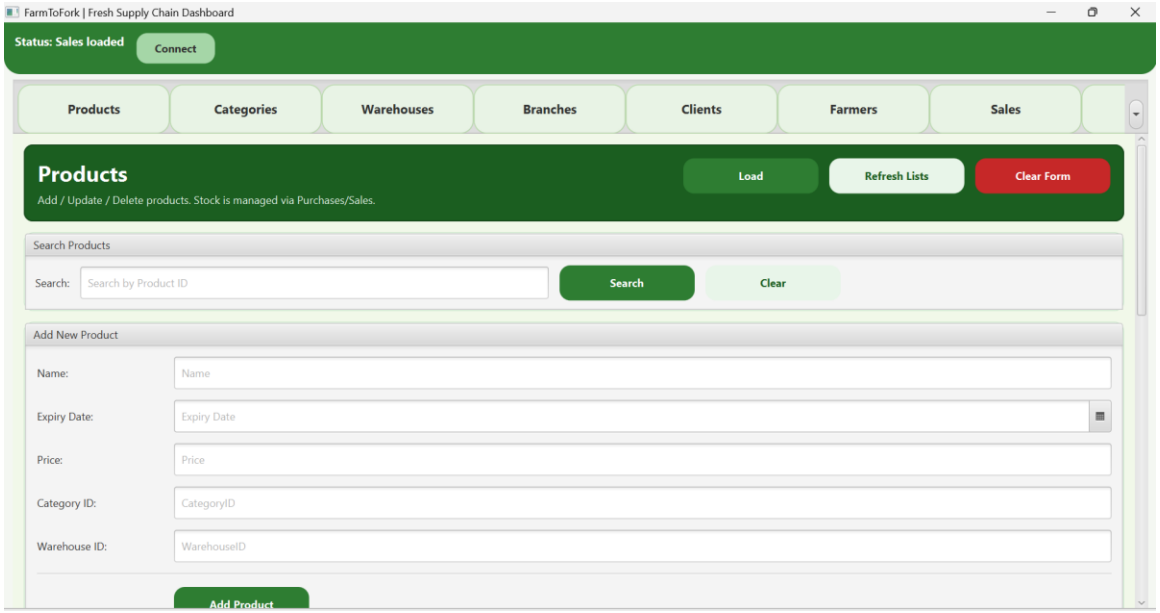
6.1 Technologies Used

The FarmToFork system was implemented using a combination of modern database and application development technologies. The user interface was developed using JavaFX, providing an interactive and user-friendly graphical interface. The database layer was implemented using MySQL, while JDBC (Java Database Connectivity) was used to establish communication between the application and the database. This technology stack ensures reliability, scalability, and ease of maintenance.

6.2 System Architecture

The system follows a layered architecture that separates concerns and improves maintainability:

- Presentation Layer (UI Layer):
  Implemented using JavaFX, this layer is responsible for user interaction. It includes forms, tables, buttons, and tabs that allow users to perform CRUD operations and view reports.



- Data Access Layer (DAO Layer):
  Each database table is managed through a dedicated Data Access Object (DAO) class that encapsulates all database operations related to that entity. For example, the **ProductDAO** class is responsible for handling all CRUD operations on the Product

table, including inserting new products, updating product details, retrieving product records, and deleting products. This design approach improves code modularity, enhances maintainability, and separates database logic from the user interface layer.

- Database Layer:
  The MySQL database stores all persistent data, including master data (products, categories, warehouses, branches, clients, and farmers) and transactional data (sales and purchases).

6.3 CRUD Operations

The system supports full CRUD (Create, Read, Update, Delete) functionality for all master entities:

- Categories
- Warehouses
- Branches
- Products
- Clients
- Farmers

Data is displayed using table views, and user input is validated before database operations are executed to ensure data integrity and prevent invalid entries.

6.4 Transaction Management

Sales and purchase operations are implemented as transactional processes to ensure data consistency:

- Purchase Transactions:
  When a purchase is recorded, a new entry is inserted into the Purchase table, and the corresponding product quantity is increased accordingly.
- Sale Transactions:
  When a sale is recorded, a new entry is inserted into the Sale table, and the product quantity is decreased. The system prevents selling quantities greater than the available stock.

These operations are executed using transactional logic to ensure that inventory updates and transaction records remain consistent.

6.5 Data Integrity and Validation

Several mechanisms were implemented to preserve data integrity:

- Validation of input values such as quantity and price to ensure they are positive.
- Enforcement of referential integrity using primary key and foreign key constraints.
- Application-level checks to prevent invalid operations, such as selling unavailable products.

6.6 Reports Implementation

The system includes a dedicated Reports module implemented using predefined SQL queries. These queries retrieve aggregated and analytical information from the database, such as sales statistics, inventory status, and revenue summaries. The reports are executed through the DAO layer and displayed directly in the user interface.

6.7 Summary

The implementation of the FarmToFork system successfully integrates the database design with a functional application. The use of a layered architecture, transactional logic, and validated user input ensures data consistency, system reliability, and ease of use. The final system fulfills all functional requirements defined in the earlier phases of the project.

7. Database Implementation (SQL)

Final SQL schema used in the project:

```
CREATE DATABASE IF NOT EXISTS farmtofork;

USE farmtofork;


SET FOREIGN_KEY_CHECKS = 0;


DROP TABLE IF EXISTS Sale;

DROP TABLE IF EXISTS Purchase;

DROP TABLE IF EXISTS Product;

DROP TABLE IF EXISTS Client;

DROP TABLE IF EXISTS Farmer;

DROP TABLE IF EXISTS Branch;

DROP TABLE IF EXISTS Warehouse;

DROP TABLE IF EXISTS Category;


SET FOREIGN_KEY_CHECKS = 1;


/* =========================

   Master tables

   ========================= */


CREATE TABLE Category (

  categoryID INT NOT NULL AUTO_INCREMENT,

  name VARCHAR(100) NOT NULL,

  description VARCHAR(200),

  PRIMARY KEY (categoryID)

);
```

```
CREATE TABLE Warehouse (
  warehouseID INT NOT NULL AUTO_INCREMENT,
  location VARCHAR(150) NOT NULL,
  capacity INT NOT NULL,
  PRIMARY KEY (warehouseID)
);

CREATE TABLE Branch (
  branchID INT NOT NULL AUTO_INCREMENT,
  name VARCHAR(100) NOT NULL,
  city VARCHAR(100),
  address VARCHAR(200),
  phone VARCHAR(30),
  PRIMARY KEY (branchID)
);

CREATE TABLE Client (
  clientID INT NOT NULL AUTO_INCREMENT,
  name VARCHAR(100) NOT NULL,
  type VARCHAR(50),
  contactInfo VARCHAR(200),
  PRIMARY KEY (clientID)
);



CREATE TABLE Farmer (
  farmerID INT NOT NULL AUTO_INCREMENT,
  name VARCHAR(100) NOT NULL,
```

```
  region VARCHAR(100),
  contactInfo VARCHAR(200),
  PRIMARY KEY (farmerID)
);

/* ========================
   Product
   ======================== */
CREATE TABLE Product (
  productID INT NOT NULL AUTO_INCREMENT,
  name VARCHAR(100) NOT NULL,
  expiryDate DATE,
  price DECIMAL(10,2) NOT NULL,
  quantity INT NOT NULL DEFAULT 0,
  categoryID INT NOT NULL,
  warehouseID INT NOT NULL,
  PRIMARY KEY (productID),
  FOREIGN KEY (categoryID) REFERENCES Category(categoryID),
  FOREIGN KEY (warehouseID) REFERENCES Warehouse(warehouseID)
);

/* ========================
   Purchase
   ======================== */
CREATE TABLE Purchase (
  purchaseID INT NOT NULL AUTO_INCREMENT,
  purchaseDate DATE NOT NULL,
  quantity INT NOT NULL,
  unitPrice DECIMAL(10,2) NOT NULL,
```

```
  totalCost DECIMAL(10,2) NOT NULL,
  warehouseEntryDate DATE,
  farmerID INT NOT NULL,
  productID INT NOT NULL,
  branchID INT NOT NULL,
  PRIMARY KEY (purchaseID),
  FOREIGN KEY (farmerID) REFERENCES Farmer(farmerID),
  FOREIGN KEY (productID) REFERENCES Product(productID),
  FOREIGN KEY (branchID) REFERENCES Branch(branchID)
);

/* ========================
   Sale (matches SaleDAO + ReportDAO)
   ======================== */
CREATE TABLE Sale (
  saleID INT NOT NULL AUTO_INCREMENT,
  saleDate DATE NOT NULL,
  quantity INT NOT NULL,
  unitPrice DECIMAL(10,2) NOT NULL,
  totalRevenue DECIMAL(10,2) NOT NULL,
  clientID INT NOT NULL,
  productID INT NOT NULL,
  branchID INT NOT NULL,
  PRIMARY KEY (saleID),
  FOREIGN KEY (clientID) REFERENCES Client(clientID),
  FOREIGN KEY (productID) REFERENCES Product(productID),
  FOREIGN KEY (branchID) REFERENCES Branch(branchID)
);
```

8. Required SQL Queries (20 Queries)

The Reports tab implements twenty analytical SELECT queries. Each query is listed below and can be tested either inside MySQL Workbench or by running it through the JavaFX Reports screen.

1) Sales per branch per month

```
SELECT b.name AS branch,
       DATE_FORMAT(s.saleDate, '%Y-%m') AS month,
       SUM(s.totalRevenue) AS total_sales
FROM Sale s
JOIN Branch b ON b.branchID = s.branchID
GROUP BY b.name, DATE_FORMAT(s.saleDate, '%Y-%m')
ORDER BY month, branch
```

2) Total revenue per client

```
SELECT c.clientID,
       c.name AS client_name,
       SUM(s.totalRevenue) AS total_revenue
FROM Sale s
JOIN Client c ON c.clientID = s.clientID
GROUP BY c.clientID, c.name
ORDER BY total_revenue DESC
```

3) Most in-demand products (by sold quantity)

```
SELECT p.productID,
       p.name AS product_name,
       SUM(s.quantity) AS total_sold
FROM Sale s
JOIN Product p ON p.productID = s.productID
GROUP BY p.productID, p.name
ORDER BY total_sold DESC
```

4) Total purchases per month

```
SELECT DATE_FORMAT(p.purchaseDate, '%Y-%m') AS month,
       SUM(p.totalCost) AS total_spending,
       SUM(p.quantity) AS total_quantity
FROM Purchase p
GROUP BY DATE_FORMAT(p.purchaseDate, '%Y-%m')
ORDER BY month
```

5) Low stock products (qty < 10)

```
SELECT productID, name, quantity
FROM Product
WHERE quantity < ?
ORDER BY quantity ASC
```

6) Products nearing expiry (within 7 days)

```
SELECT productID, name, expiryDate
FROM Product
WHERE expiryDate IS NOT NULL
  AND expiryDate <= (CURDATE() + INTERVAL ? DAY)
ORDER BY expiryDate ASC
```

7) Total revenue per branch

```
SELECT b.branchID,
       b.name AS branch_name,
       SUM(s.totalRevenue) AS total_revenue
FROM Sale s
JOIN Branch b ON b.branchID = s.branchID
GROUP BY b.branchID, b.name
ORDER BY total_revenue DESC
```

8) Total sold quantity per product

```
SELECT p.productID,
       p.name AS product_name,
       SUM(s.quantity) AS total_sold
FROM Sale s
JOIN Product p ON p.productID = s.productID
GROUP BY p.productID, p.name
ORDER BY total_sold DESC
```

9) Total revenue per product

```
SELECT p.productID,
       p.name AS product_name,
       SUM(s.totalRevenue) AS revenue
FROM Sale s
JOIN Product p ON p.productID = s.productID
GROUP BY p.productID, p.name
ORDER BY revenue DESC
```

10) Products never sold

```
SELECT p.productID,
       p.name AS product_name,
       p.quantity AS current_qty
FROM Product p
LEFT JOIN Sale s ON s.productID = p.productID
WHERE s.saleID IS NULL
ORDER BY p.productID DESC
```

11) Clients with no sales

```
SELECT c.clientID,
       c.name AS client_name,
       c.type AS client_type
FROM Client c
LEFT JOIN Sale s ON s.clientID = c.clientID
WHERE s.saleID IS NULL
ORDER BY c.clientID DESC
```

12) Monthly sales totals (revenue + qty)

```
SELECT DATE_FORMAT(s.saleDate,'%Y-%m') AS month,
       SUM(s.totalRevenue) AS revenue,
       SUM(s.quantity) AS total_qty
FROM Sale s
GROUP BY DATE_FORMAT(s.saleDate,'%Y-%m')
ORDER BY month
```

13) Monthly profit estimate (Sales - Purchases)

```
SELECT m.month AS month,
           COALESCE(sa.sales_revenue,0) AS sales_revenue,
           COALESCE(pu.purchase_cost,0) AS purchase_cost,
           (COALESCE(sa.sales_revenue,0) - COALESCE(pu.purchase_cost,0)) AS
profit_estimate
    FROM (
        SELECT DATE_FORMAT(d,'%Y-%m') AS month
        FROM (
            SELECT saleDate AS d FROM Sale
            UNION
            SELECT purchaseDate AS d FROM Purchase
        ) x
    ) m
    LEFT JOIN (
        SELECT DATE_FORMAT(saleDate,'%Y-%m') AS month,
               SUM(totalRevenue) AS sales_revenue
        FROM Sale
        GROUP BY DATE_FORMAT(saleDate,'%Y-%m')
```

```
    ) sa ON sa.month = m.month
    LEFT JOIN (
        SELECT DATE_FORMAT(purchaseDate,'%Y-%m') AS month,
               SUM(totalCost) AS purchase_cost
        FROM Purchase
        GROUP BY DATE_FORMAT(purchaseDate,'%Y-%m')
    ) pu ON pu.month = m.month
    ORDER BY m.month
""";
```

14) Average selling price per product

```
SELECT p.productID,
       p.name AS product_name,
       AVG(s.unitPrice) AS avg_selling_price
FROM Sale s
JOIN Product p ON p.productID = s.productID
GROUP BY p.productID, p.name
ORDER BY avg_selling_price DESC
```

15) Average purchase price per product

```
SELECT pr.productID,
       pr.name AS product_name,
       AVG(p.unitPrice) AS avg_purchase_price
FROM Purchase p
JOIN Product pr ON pr.productID = p.productID
GROUP BY pr.productID, pr.name
ORDER BY avg_purchase_price DESC
```

16) Top clients by purchased quantity

```
SELECT c.clientID,
       c.name AS client_name,
       SUM(s.quantity) AS total_qty
FROM Sale s
JOIN Client c ON c.clientID = s.clientID
GROUP BY c.clientID, c.name
ORDER BY total_qty DESC
```

17) Top farmers by supplied quantity

```
SELECT f.farmerID,
       f.name AS farmer_name,
       SUM(p.quantity) AS total_supplied
FROM Purchase p
JOIN Farmer f ON f.farmerID = p.farmerID
GROUP BY f.farmerID, f.name
ORDER BY total_supplied DESC
```

18) Purchases per branch per month

```
SELECT b.name AS branch,
       DATE_FORMAT(p.purchaseDate, '%Y-%m') AS month,
       SUM(p.totalCost) AS total_spending
FROM Purchase p
JOIN Branch b ON b.branchID = p.branchID
GROUP BY b.name, DATE_FORMAT(p.purchaseDate, '%Y-%m')
ORDER BY month, branch
```

19) Sales per product per branch

```
SELECT b.name AS branch,
       pr.name AS product,
       SUM(s.quantity) AS total_sold
FROM Sale s
JOIN Branch b ON b.branchID = s.branchID
JOIN Product pr ON pr.productID = s.productID
GROUP BY b.name, pr.name
ORDER BY branch, total_sold DESC
```

20) Stock value per warehouse

```
SELECT w.warehouseID,
       w.location AS warehouse,
       SUM(pr.price * pr.quantity) AS stock_value
FROM Product pr
JOIN Warehouse w ON w.warehouseID = pr.warehouseID
GROUP BY w.warehouseID, w.location
ORDER BY stock_value DESC
```

9. Demo Scenario

This section presents a demonstration scenario that illustrates the main functionalities of the FarmToFork system and highlights its core features during execution.

Step 1: Adding Categories and Products

The demonstration begins by adding a new product category through the Categories management interface. After that, a new product is added and assigned to the previously created category and an existing warehouse. Product details such as name, price, expiry date, and initial quantity are entered. The product is then displayed in the products table, confirming successful insertion.

Step 2: Adding Farmers and Clients

Next, a new farmer is added to the system by providing basic information such as name, region, and contact details. Similarly, a new client is added by specifying the client's name, type, and contact information. Both records appear immediately in their respective tables, demonstrating the CRUD functionality of the system.

Step 3: Recording a Purchase (Stock Increase)

A purchase transaction is then recorded by selecting a farmer, product, and branch, and entering the purchased quantity. Once the purchase is confirmed, a new record is added to the Purchase table. At the same time, the system automatically updates the product quantity by increasing the available stock. This step demonstrates transaction handling and inventory update consistency.

Step 4: Recording a Sale (Stock Decrease)

A sale transaction is performed by selecting a client, product, and branch, and specifying the sold quantity. The system validates that sufficient stock is available before completing the sale. After confirmation, the sale record is stored in the Sale table, and the product quantity is decreased accordingly. This ensures that the inventory accurately reflects real-time sales operations.

Step 5: Monitoring Inventory Status

The system is then used to monitor inventory status by identifying products with low stock levels and products nearing their expiry dates. These views help demonstrate how the

system supports inventory control and reduces product waste by enabling timely decision-making.

Step 6: Generating Reports

Finally, the Reports module is accessed to generate analytical reports. Examples include total sales per branch, revenue per client, and inventory status reports. These reports demonstrate the system's ability to provide summarized and analytical insights using SQL queries, supporting managerial decision-making.

Demo Summary

This demo scenario demonstrates the complete workflow of the FarmToFork system, from managing master data to executing transactions and generating reports. It highlights the integration between the database design and the application logic, as well as the system's ability to maintain data consistency and support real-world operational needs.

10. Conclusion

The FarmToFork system successfully delivers a complete database-driven solution that supports the operational needs of GreenRoots Cooperative. Through a well-structured database design, normalized relational schema, and a fully functional JavaFX application, the system enables efficient management of products, inventory, purchases, sales, and analytical reporting. The integration of transactional logic ensures data consistency, while the reporting module provides valuable insights for decision-making. Overall, the project fulfills all functional and academic requirements defined across the project phases and demonstrates the effective application of database design principles in a real-world scenario.

11.Future Work

Several enhancements can be considered to further extend the system's functionality. These include implementing user authentication and role-based access control, adding database triggers to automate inventory updates, and supporting barcode or QR code integration for faster product handling. Additional analytical reports and dashboard visualizations could also be introduced to enhance decision support. Finally, optimizing performance for large datasets and enabling multi-warehouse inventory tracking would further improve system scalability and usability.